9.1 多Agent配置与路由

> **生成模型**：GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~35k tokens，输出 ~4k tokens（估算，本节）

很多人刚接触 OpenClaw 时，脑子里默认还是“一个 AI 聊天助手”。这没问题，起步阶段确实该先这么用。但只要你开始把它放进真实生活或者真实项目里，很快就会撞到一个很具体的问题：同一个 Agent 既要陪你聊天、处理私人消息，又要改代码、查资料、跑命令，最后往往什么都能碰一点，但什么都不够稳。

多 Agent 的意义，不在“更高级”，而在“把角色拆清楚”。工作上的消息交给工作 Agent，私人提醒交给 personal Agent，真正能动代码、读仓库、跑测试的事情交给 code Agent。这样一拆，整个系统马上顺了：上下文不容易串，权限边界更好控，模型也可以按任务分配，不用所有事都扔给同一个最贵模型。

> **衍生解释：路由**
>
> 这里的“路由”，可以把它理解成快递分拣。消息先进总仓库，然后系统根据规则决定：这条该送给谁。规则可以按消息来自哪个通道、哪个账号、哪个联系人或群组来写。路由配得清楚，多 Agent 才真的好用。

9.1.1 为什么单 Agent 容易越用越乱

单 Agent 最大的问题不是智力不够，而是职责太杂。比如你上午让它帮你 review PR，中午又让它记住晚上去拿快递，下午再让它去查某个 SDK 的文档差异。技术上下文、生活上下文、外部网页碎片全混在一起，模型后面很容易“带着上一题的味道”处理下一题。

还有两个问题也很现实。

第一是权限。你不一定希望“私人聊天助手”能跑 `bash`，也不一定希望“代码代理”能去所有社交通道主动发消息。角色一旦分开，每个 Agent 的工具允许范围就能单独收紧。

第二是成本。研究类任务适合长上下文、网页检索强的模型；写代码适合编码能力稳定的模型；日常聊天和提醒反而可以用更便宜的模型。多 Agent 之后，模型选型终于能按岗位来，而不是一刀切。

9.1.2 `agents.list[]`： 多 Agent 的入口

OpenClaw 里，多 Agent 的主入口就是 `openclaw.json` 里的 `agents.list[]` 。你可以把它当成一张“岗位表”。列表里的每一项，都是一个独立 Agent：有自己的 `id` 、名字、模型、技能、工作目录，必要时还可以有单独的工具策略。

最实用的思路不是一上来配七八个 Agent，而是先配三个：

- `work`： 处理工作沟通、任务整理、日程提醒。
- `personal`： 处理私人消息、生活类记录、轻量问答。
- `code`： 专门服务代码仓库，允许读写文件、跑测试、查文档。

一个适合起步的完整配置可以写成这样：

```jsonc
{
  agents: {
    defaults: {
      model: "anthropic/claude-sonnet-4-5",
      workspace: "~/openclaw-workspace",
      skills: ["memory", "calendar"],
      tools: {
        deny: ["bash"]
      }
    },
    list: [
      {
        id: "work",
        name: "Work Agent",
        model: "openai/gpt-5.2",
        skills: ["calendar", "gmail", "slack"],
        workspace: "~/work"
      },
      {
        id: "personal",
        name: "Personal Agent",
        model: "openai/gpt-4o-mini",
        skills: ["memory", "weather"],
        workspace: "~/personal"
      },
      {
        id: "code",
        name: "Code Agent",
        model: "anthropic/claude-sonnet-4-5",
        skills: ["coding-agent", "github", "browser"],
        workspace: "~/projects",
        tools: {
          allow: ["read", "write", "edit", "bash", "browser", "sessions_*"],
          deny: ["send"]
        }
      }
    ]
  }
}
```

这里有个很关键的点： `defaults` 是“公共底板”， `list[]` 里的配置再按 Agent 覆盖。这样你不用在每个 Agent 上重复写一堆相同配置，但又能给 `code` 单独放开工具权限。

9.1.3 路由规则：按 channel、account、peer 分工

光把 Agent 定义出来还不够，接下来要解决“消息到底进谁的会话”。这件事靠 `bindings` 。

OpenClaw 里最常见的三种路由方式是：按通道、按账号、按对端。

按通道路由

这个最直观。比如 Discord 全部走 `code` ，Slack 全部走 `work` 。适合“这个通道本身用途就很明确”的情况。

```
{
  bindings: [
    {
      agentId: "code",
      match: { channel: "discord", accountId: "*" }
    },
    {
      agentId: "work",
      match: { channel: "slack", accountId: "*" }
    }
  ]
}
```

按账号路由

如果你在同一个通道里挂了多个账号，这种方式就很好用。比如 Telegram 上有一个工作机器人账号和一个私人机器人账号，它们都接进 OpenClaw，但分别分配给不同 Agent。

```
{
  bindings: [
    {
      agentId: "work",
      match: { channel: "telegram", accountId: "tg-work-bot" }
    },
    {
      agentId: "personal",
      match: { channel: "telegram", accountId: "tg-life-bot" }
    }
  ]
}
```

按对端路由

“对端”就是消息的具体对象，可能是某个联系人，也可能是某个群。这个特别适合把少数关键群聊单独交给专业 Agent。比如技术群给 `code` ，家人群给 `personal` 。

```
{
  bindings: [
    {
      agentId: "code",
      match: {
        channel: "telegram",
        peer: { kind: "group", id: "-1002003004001" }
      }
    },
    {
      agentId: "personal",
      match: {
        channel: "telegram",
        peer: { kind: "group", id: "-1002003004999" }
      }
    }
  ]
}
```

这三种规则往往会混着用。简单说：channel 适合粗分流，account 适合多身份，peer 适合精准指定。

9.1.4 Agent 专属模型、技能、工具，别图省事全开

多 Agent 真正好用，靠的是差异化配置，而不是“复制三份同样的 Agent”。

先说模型。 `work` 更适合总结、沟通、安排； `personal` 更看重成本和响应速度； `code` 则要偏编码稳定性和长上下文能力。模型不是越强越好，而是岗位匹配最好。

再说技能。工作 Agent 可以挂邮箱、日历、Slack；personal Agent 只保留记忆、天气、提醒；code Agent 再给浏览器、GitHub、编码技能。技能一少，上下文噪音也会少。

最后是工具权限，这一项经常决定“系统稳不稳”。一个保守但靠谱的原则是：

- 能不给 `bash` 就不给。
- 能不给跨会话通信就先不给。
- 能不给主动发消息能力就别默认放开。

换句话说，多 Agent 不是为了让所有 Agent 都变成全能超人，而是为了让每个 Agent 只干它擅长、也该被允许干的那一段。

9.1.5 一个真实能用的例子：工作 + 私人 + 代码三分法

下面这个例子，已经足够让一个学生或者开发者把 OpenClaw 从“单聊天机器人”升级成“多岗位 AI 团队”。

```
{
  agents: {
    defaults: {
      workspace: "~/openclaw-workspace",
      model: "openai/gpt-4o-mini",
      tools: {
        deny: ["bash"]
      }
    },
    list: [
      {
        id: "work",
        name: "Work Agent",
        model: "openai/gpt-5.2",
        skills: ["calendar", "gmail", "slack"],
        workspace: "~/work"
      },
      {
        id: "personal",
        name: "Personal Agent",
        skills: ["memory", "weather"],
        workspace: "~/personal"
      },
      {
        id: "code",
        name: "Code Agent",
        model: "anthropic/claude-sonnet-4-5",
        skills: ["coding-agent", "github", "browser"],
        workspace: "~/projects",
        tools: {
          allow: ["read", "write", "edit", "bash", "browser", "sessions_list", "session
          deny: ["send"]
        }
      }
    ]
  },
  bindings: [
    {
      agentId: "work",
      match: { channel: "slack", accountId: "*" }
    },
    {
      agentId: "personal",
```

```
      match: { channel: "whatsapp", accountId: "*" }
    },
    {
      agentId: "code",
      match: { channel: "discord", accountId: "*" }
    },
    {
      agentId: "code",
      match: {
        channel: "telegram",
        peer: { kind: "group", id: "-1001234567890" }
      }
    }
  ]
}
```

这套配置有几个明显好处。

第一，工作和生活上下文被切开了，不会出现“上午写周报，晚上问晚饭吃什么，结果记忆和风格全混了”的情况。

第二，代码 Agent 的权限比另外两个大，但它的入口更窄，只在开发相关通道和群组触发。这样危险能力不是彻底没了，而是被关在更合理的边界里。

第三，这种结构后面很好扩展。等你熟了，再加 `research` 或 `ops` 都很自然。

9.1.6 配置时最容易踩的坑

第一个坑是想一步到位，结果配了太多 Agent。岗位太细，最后自己都记不住哪个负责什么。建议先 3 个，再慢慢长。

第二个坑是只建 Agent，不写路由。这样消息还是全落到默认 Agent，上面那些配置等于白写。

第三个坑是复制粘贴同一份技能和工具集。这样虽然表面叫多 Agent，实际上还是一个大杂烩，只是换了几个名字。

第四个坑是把 peer 规则写得过细。你可以这么做，但维护成本会很高。多数时候，先按 channel 或 account 粗分，再对少数关键群组加 peer 例外，就够了。

本节小结

多 Agent 的核心价值，不是“同时跑多个模型”，而是把不同工作流拆成不同岗位。 `agents.list[]` 负责把岗位定义出来， `bindings` 负责把消息送到正确的人手里。最实用的起步方式，就是先做 `work` 、 `personal` 、 `code` 三个 Agent，再按 channel、account、peer 三层规则分流。这样一来，OpenClaw 就不再只是一个会聊天的 AI，而开始像一个分工明确的小团队。下一节我们继续看更动态的一层：主 Agent 怎么拉起子代理，以及多个 Agent 之间到底怎么传话。